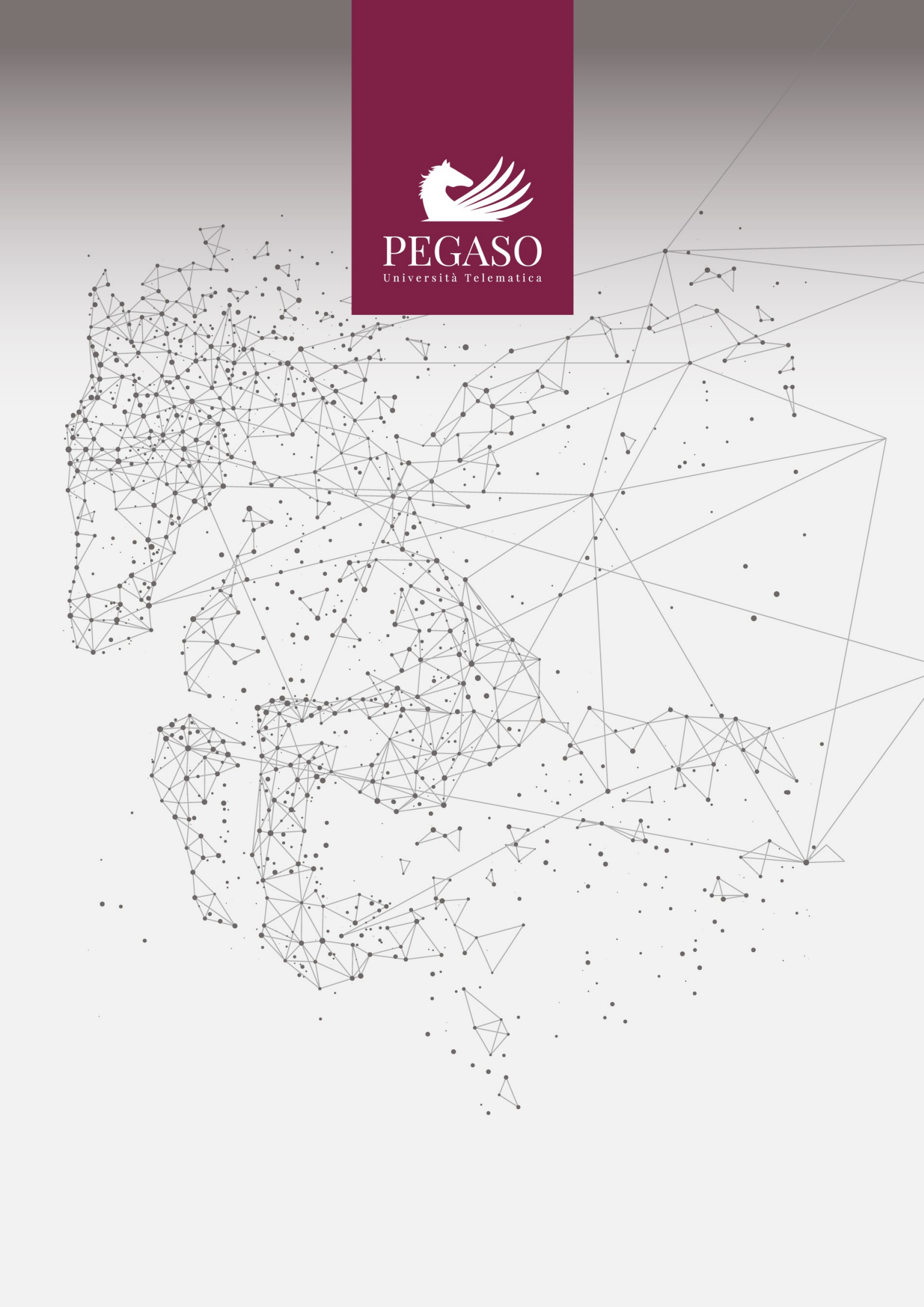

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. WRAPPER 3
2. INTERCETTORI 5
3. MIDDLEWARE MODIFICABILE 8
BIBLIOGRAFIA 10

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Wrapper

Quando si costruisce un sistema distribuito a partire da componenti esistenti, ci si imbatte immediatamente in un problema fondamentale: le interfacce offerte dal componente legacy probabilmente non sono adatte per tutte le applicazioni. Questo può creare difficoltà nell'integrare questi componenti in un sistema distribuito e rendere difficile l'accesso alle funzionalità offerte dai componenti da parte delle applicazioni client.

Per risolvere questo problema, si può utilizzare un wrapper o un adattatore. Un wrapper è un componente speciale che offre un'interfaccia accettabile per un'applicazione client, le cui funzioni vengono trasformate in quelle disponibili nel componente. In questo modo, il wrapper o l'adattatore funge da intermediario tra il componente e l'applicazione client, consentendo all'applicazione di accedere alle funzionalità del componente attraverso un'interfaccia compatibile.

Seppur originariamente definiti strettamente nel contesto della programmazione orientata agli oggetti, i wrapper hanno un ruolo importante anche nel contesto dei sistemi distribuiti. Ad esempio, un adattatore di oggetti è un componente che consente alle applicazioni di invocare oggetti remoti, anche se questi ultimi sono stati implementati come una combinazione di funzioni di libreria che operano sulle tabelle di un database relazionale. In questo caso, l'adattatore di oggetti funge da intermediario tra l'applicazione e l'oggetto remoto, consentendo all'applicazione di invocare le funzionalità dell'oggetto attraverso un'interfaccia compatibile.

Un altro esempio di utilizzo dei wrapper nei sistemi distribuiti è il servizio di storage S3 di Amazon. Questo servizio offre due tipi di interfacce, una che segue un'architettura RESTful e l'altra che segue un approccio più tradizionale. Per l'interfaccia RESTful, i client utilizzeranno il protocollo HTTP, comunicando essenzialmente con un server Web tradizionale che in questo caso agisce come adattatore per il servizio di storage effettivo, smembrando parzialmente le richieste in arrivo e consegnandole successivamente a server specializzati interni a S3. Più specificamente, il server Web funge da wrapper per il servizio di storage, consentendo ai client di accedere alle funzionalità del servizio attraverso un'interfaccia compatibile con il protocollo HTTP.

I wrapper hanno sempre svolto un ruolo importante nell'estendere i sistemi con componenti ormai obsoleti. L'estensibilità è fondamentale per raggiungere l'apertura dei sistemi, e i wrapper possono essere utilizzati per garantire che le applicazioni possano accedere alle funzionalità dei componenti esistenti anche se le loro interfacce non sono compatibili. Tuttavia, l'approccio di sviluppare un wrapper specifico per ogni applicazione non è scalabile, poiché con N applicazioni sarebbero necessari N × (N - 1) = O(N^2) wrapper.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per facilitare la riduzione del numero di wrapper necessari, si può utilizzare il middleware. Un modo per farlo è implementare un cosiddetto broker, che è logicamente un componente centralizzato che gestisce gli accessi tra diverse applicazioni. Un tipo spesso utilizzato è un broker di messaggi. In questo caso, le applicazioni inviano semplicemente richieste al broker contenenti informazioni su ciò di cui hanno bisogno. Il broker, avendo conoscenza di tutte le applicazioni rilevanti, contatta le applicazioni appropriate, combina e trasforma eventualmente le risposte e restituisce il risultato all'applicazione iniziale. In questo modo, poiché il broker offre un'unica interfaccia per ogni applicazione, sono necessari al massimo 2N = O(N) wrapper invece di O(N^2).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Intercettori

Un intercettore, nel senso più ampio, non è altro che un costrutto software; il suo unico scopo è quello di deviare il normale flusso di controllo e fare spazio per l'esecuzione di altro codice (specifico dell'applicazione). Gli intercettori sono uno degli strumenti più importanti per personalizzare il middleware per soddisfare i requisiti di una particolare applicazione. Di conseguenza, svolgono un ruolo significativo nel processo di apertura del middleware. Rendere generici gli intercettori può richiedere uno sforzo significativo per l'implementazione, e non è chiaro se, in tali circostanze, la generalità debba essere preferita all'applicabilità e alla semplicità. Inoltre, disporre solo di strutture di intercettazione limitate si tradurrà, in molte circostanze, in una migliore gestione del software e del sistema distribuito nel suo complesso.

Si consideri, ad esempio, il fatto che molti sistemi distribuiti basati su oggetti includono il supporto per l'intercettazione nei loro set di funzionalità. Il concetto alla base è semplice: un oggetto A può invocare un metodo associato a un oggetto B, anche se l'oggetto B risiede su una macchina distinta dall'oggetto A. L'invocazione di un oggetto remoto si compie attraverso le seguenti tre fasi:

1. L'oggetto A è dotato di un'interfaccia locale identica sotto ogni aspetto all'interfaccia che l'oggetto B mette a disposizione dei suoi utenti. A effettua una chiamata al metodo fornito da tale interfaccia.
2. La chiamata effettuata da A viene trasformata in una chiamata di oggetto di tipo generico. Ciò è reso possibile da un'interfaccia generale di chiamata degli oggetti fornita dal middleware sulla macchina in cui risiede A.
3. Infine, l'invocazione dell'oggetto generico viene convertita in un messaggio, che viene quindi trasmesso tramite l'interfaccia di rete a livello di trasporto fornita dal sistema operativo locale di A.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La figura 1 illustra questo schema nella sua interezza.

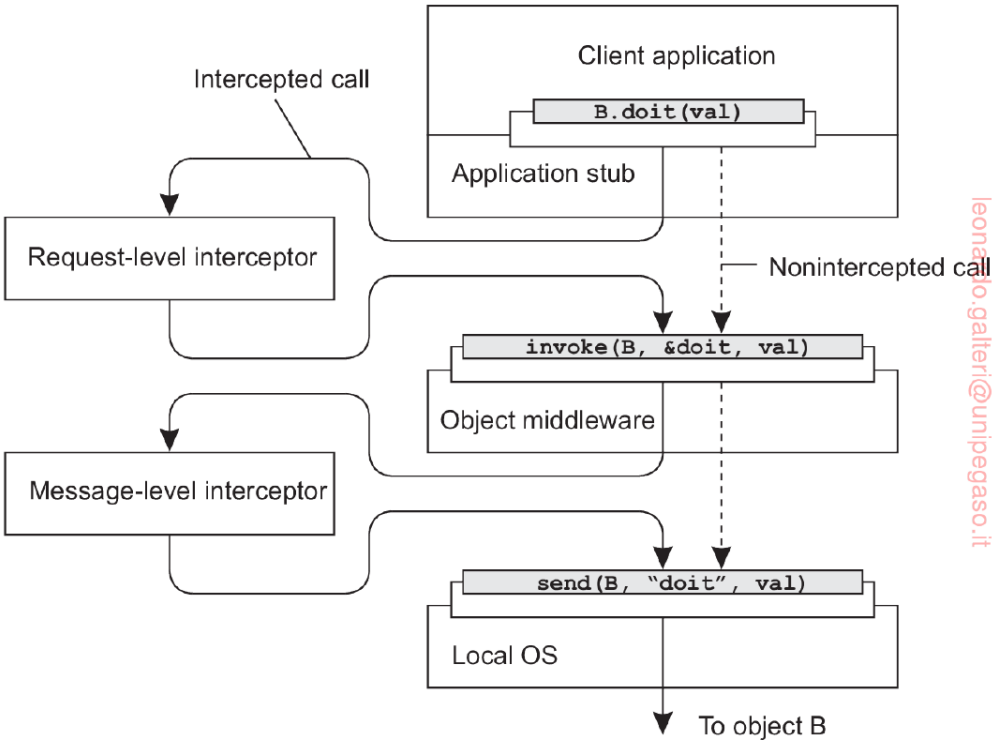


Figura 1 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

Dopo il primo passaggio, la chiamata B.doit(val) viene convertita in una chiamata generica come invoke(B, &doit, val), che include un riferimento al metodo di B oltre ai parametri passati insieme alla chiamata. Ora supponiamo che l'oggetto B venga copiato più volte. In tal caso, la pratica migliore è quella di richiamare effettivamente ogni replica. A questo punto, è decisamente chiaro che c'è bisogno di un intecettore. Il compito dell'intercettore a livello di richiesta consiste nell'eseguire operazioni di chiamata del metodo invoke(B, &doit, val) per ciascuna delle repliche. La cosa più positiva di tutto questo è che l'oggetto A non deve essere consapevole della replica dell'oggetto B e il middleware dell'oggetto non deve avere componenti speciali che gestiscono le chiamate replicate. L'unico componente che deve avere familiarità con la replica di B è l'intercettore a livello di richiesta, che può essere aggiunto al middleware.

Una chiamata che deve essere effettuata a un oggetto remoto dovrà infine essere trasmessa sulla rete. In realtà, questo indica che l'interfaccia di messaggistica fornita dal sistema operativo di vicinato dovrà essere attivata per procedere. A questo livello, un intercettore a livello di messaggio potrebbe essere in grado di passare la chiamata all'oggetto di destinazione. Immaginiamo, che il parametro *val* corrisponda effettivamente a una matrice molto ampia di dati. In questo scenario, potrebbe essere prudente

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

suddividere i dati in parti più piccole prima di inviarli in modo che possano essere riassemblati a destinazione. È possibile che le prestazioni o l'affidabilità migliorino con tale frammentazione. Ancora una volta, il middleware non ha bisogno di essere consapevole di questa frammentazione perché l'intercettore di livello inferiore gestirà il resto della comunicazione con il sistema operativo locale in modo trasparente.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Middleware modificabile

I wrapper e gli intercettori forniscono un mezzo per estendere e personalizzare la funzionalità del middleware. A causa della natura dinamica dell'ambiente in cui vengono eseguite le applicazioni distribuite, è necessario un adattamento per soddisfare i requisiti di tali applicazioni. Le alterazioni includono, ma non sono limitate a, quelle causate dalla mobilità, una variazione significativa nella qualità del servizio fornito dalle reti, hardware difettoso e consumo della batteria. Invece di attribuire la responsabilità di rispondere alle modifiche alle applicazioni stesse, questo compito viene assegnato al middleware. Inoltre, le dimensioni di un sistema distribuito rendono sempre più difficile modificare i suoi componenti senza interrompere definitivamente il funzionamento del sistema. La capacità di apportare modifiche al volo è esattamente ciò che è richiesto.

A causa di queste influenze significative dall'ambiente circostante, un numero significativo di progettisti middleware sta prendendo in considerazione la costruzione di software adattivo. Il middleware potrebbe non solo aver bisogno di essere adattivo, ma che dovremmo essere in grado di modificarlo intenzionalmente senza danneggiarlo. Questo perché il middleware potrebbe non solo dover essere adattabile, ma anche perché potrebbe essere necessario essere in grado di supportare nuove funzionalità. Se visti in questa luce, gli intercettori possono essere visti come un mezzo attraverso il quale il flusso standard di controllo può essere adattato. Un esempio di modifica di un sistema consiste nell'effettuare sostituzioni ai suoi componenti software mentre è in esecuzione. In effetti, il metodo di costruzione dinamica del middleware dalle sue parti costitutive sta rapidamente diventando una delle strategie più comuni per lo sviluppo di middleware modificabili.

L'obiettivo di supportare la modificabilità attraverso la composizione è al centro della progettazione basata su componenti. In fase di progettazione o di runtime, la configurazione di un sistema può essere impostata in modo statico o dinamico. Quest'ultimo richiede il supporto per il late binding, un metodo che è stato implementato con successo in ambienti di linguaggio di programmazione, nonché per i sistemi operativi in cui i moduli possono essere caricati e scaricati a discrezione dell'utente. Al giorno d'oggi la ricerca sulla selezione automatica della migliore implementazione di un componente durante l'esecuzione è consolidata. Tuttavia, il processo è ancora difficile per i sistemi distribuiti, in particolare se si considera il fatto che la sostituzione di un componente richiede di sapere esattamente quale sarà l'effetto di tale sostituzione sugli altri componenti. I componenti spesso non sono così autonomi come si potrebbe inizialmente credere.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il nocciolo della questione è che per supportare le modifiche dinamiche al software che compone il middleware, abbiamo bisogno almeno di un supporto fondamentale per caricare e scaricare i componenti mentre il programma è in esecuzione. Inoltre, sono necessarie specifiche esplicite delle interfacce offerte da ciascun componente e delle interfacce che richiede. Queste specifiche sono necessarie per ogni componente. Se lo stato del sistema viene mantenuto tra le chiamate a un componente, sono necessarie ulteriori precauzioni speciali. Dovrebbe essere abbastanza ovvio che organizzare il middleware in modo tale da poter essere modificato richiede molta attenzione focalizzata.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- M. van Steen and A.S. Tanenbaum, Distributed Systems, 3rd ed., distributed-systems.net, 2017.
- Schmidt A. Implicit Human Computer Interaction Through Context. Personal and Ubiquitous Computing, 2000.
- Parlavantzas N. and Coulson G. Designing and Constructing Modifiable Middleware using Component Frameworks, 2007.
- Yellin D. Competitive Algorithms for the Dynamic Selection of Component Implementations. IBM Systems Journal, 2003.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).